

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : SCALA FOR DATA SCIENCE

MODULE 2
PRACTICAL NO 1 AND 2

Aim : 1. Find the Pearson correlation between two lists of numbers. Implement the formula for Pearson correlation coefficient.

- Import dataset
- Find Pearson correlation coefficient.
- Interpret whether it is positive or negative relationship
- Infer your result using Significance and the t-test

2. Calculate the moving average of a time series data using Scala collections.

- Use dataset
- Perform required Feature Engg.
- find :simple moving average (SMA), weighted moving average (WMA), and exponential moving average (EMA).
- Use data Visualization
- Write inference / explain your result.
- Justify which type of MA is best for your Dataset

Practical 1 :

CODE -

```
import scala.io.Source
```

```
object omwala_M2_pract1 {
```

```
  def main(args: Array[String]): Unit = {
```

```
    val stream = getClass.getResourceAsStream("/fifa.csv")
```

```
    if (stream == null) {  
      println("Error: File not found in resources folder!")  
      return  
    }  
  }
```

```
  val file = Source.fromInputStream(stream)
```

```
  val data = file.getLines().drop(1).flatMap { line =>  
    val cols = line.split(",")
```

```
    for {  
      x <- cols(2).trim.toDoubleOption  
      y <- cols(3).trim.toDoubleOption  
    } yield (x, y)  
  }.toList
```

```
  file.close()
```

```
  val (x, y) = data.unzip
```

Name : Om Wala

Roll No : S119

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : SCALA FOR DATA SCIENCE

```
val n = x.length.toDouble

val meanX = x.sum / n
val meanY = y.sum / n

val num = x.zip(y).map {
  case (xi, yi) =>
    (xi - meanX) * (yi - meanY)
}.sum

val den = math.sqrt(
  x.map(xi => math.pow(xi - meanX, 2)).sum *
  y.map(yi => math.pow(yi - meanY, 2)).sum
)

val r =
  if (den == 0) 0.0
  else num / den

val relation =
  if (r > 0.7) "Strong Positive"
  else if (r > 0) "Weak Positive"
  else "Negative"

val df = n - 2

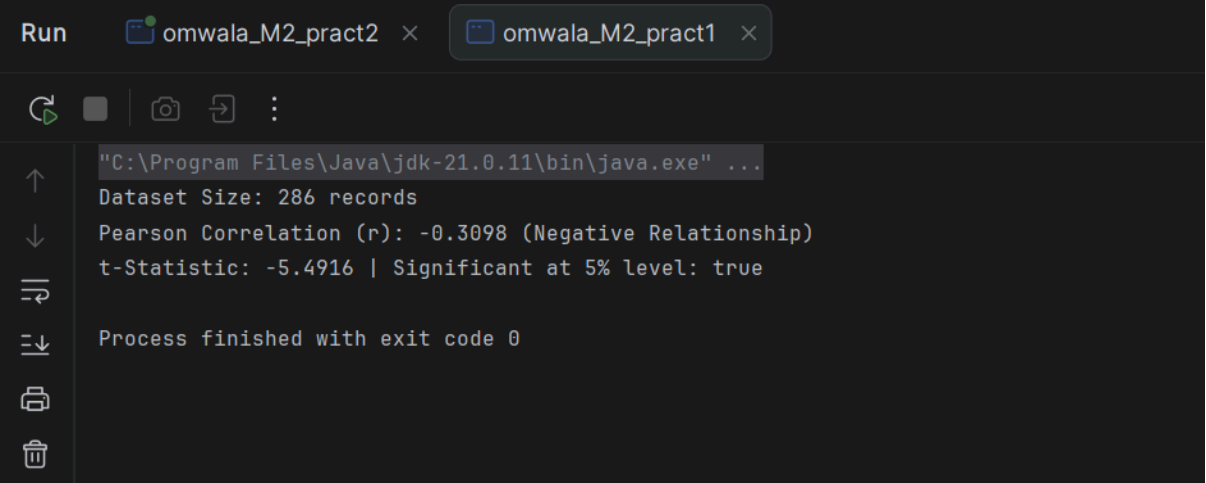
val tStat =
  if (math.abs(r) == 1.0)
    Double.PositiveInfinity
  else
    r * math.sqrt(df / (1 - r * r))

val isSignificant = math.abs(tStat) > 1.96

println(s"Dataset Size: ${n.toInt} records")
println(f"Pearson Correlation (r): $r%.4f ($relation Relationship)")
println(f"t-Statistic: $tStat%.4f | Significant at 5%% level: $isSignificant")
}
}
```

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : SCALA FOR DATA SCIENCE

OUTPUT -



```
Run omwala_M2_pract2 x omwala_M2_pract1 x
"C:\Program Files\Java\jdk-21.0.11\bin\java.exe" ...
Dataset Size: 286 records
Pearson Correlation (r): -0.3098 (Negative Relationship)
t-Statistic: -5.4916 | Significant at 5% level: true
Process finished with exit code 0
```

Practical 2

CODE -

```
import scala.io.Source
import breeze.plot._

object omwala_M2_pract2 {

  def main(args: Array[String]): Unit = {
    val stream = getClass.getResourceAsStream("/fifa.csv")

    if (stream == null) {
      println("Error: File not found in resources folder!")
      return
    }
    val file = Source.fromInputStream(stream)
    val lines = file.getLines().drop(1)
    val data = lines.flatMap { line =>
      val cols = line.split(",")
      cols(2).trim.toDoubleOption
    }.toList

    file.close()

    println("Original Data:")
    println(data.mkString(", "))

    val cleanData = data.filter(!_.isNaN)

    val window = 3

    val sma = cleanData.sliding(window).map(_._sum / window).toList

    val weights = List(1, 2, 3)
```

Name : Om Wala

Roll No : S119

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : SCALA FOR DATA SCIENCE

```
val weightSum = weights.sum.toDouble

val wma = cleanData.sliding(window).map { values =>
  values.zip(weights).map {
    case (v, w) => v * w
  }.sum / weightSum
}.toList

val alpha = 2.0 / (window + 1)

var ema = List(cleanData.head)

for (i <- 1 until cleanData.length) {
  val value = alpha * cleanData(i) + (1 - alpha) * ema.last
  ema = ema :+ value
}

println("\nSimple Moving Average")
println(sma.mkString(", "))

println("\nWeighted Moving Average")
println(wma.mkString(", "))

println("\nExponential Moving Average")
println(ema.mkString(", "))

val fig = Figure("Moving Average")
val plt = fig.subplot(0)

plt += plot((0 until cleanData.length).map(_.toDouble), cleanData)
plt += plot((window - 1 until cleanData.length).map(_.toDouble), sma)

plt += plot((window - 1 until cleanData.length).map(_.toDouble), wma)

plt += plot((0 until ema.length).map(_.toDouble), ema)

plt.title = "Moving Average Comparison"

fig.refresh()

}

}
```

OUTPUT -

